Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Puebla

TE3002B: Implementación de robótica inteligente (Gpo 501)

Profesor: Dr. Rigoberto Cerino Jiménez

**Reto semanal 2. Manchester Robotics**

Equipo *I miss NWJNS*


| | |
|---|---|
| Tania Sofia Arrazola Bello | A01738124 |
| Iñaki Ahedo Madrid | A01738231 |
| Marcos Allen Martínez Cortés | A01737939 |


12 de abril de 2026

## Resumen

En este reto semanal 2 se llevó a cabo el desarrollo de un sistema de navegación multipunto para el robot Puzzlebot utilizando el framework ROS 2. Se implementó una arquitectura basada en nodos que permite la generación y seguimiento de trayectorias definidas por el usuario, mediante el uso de mensajes personalizados y/o el uso de inputs por consola tipo trayectoria y el tópico `/cmd_vel` para el control del robot.

Durante el desarrollo se diseñó un generador de trayectorias capaz de calcular distancias y ángulos entre puntos, así como estimar velocidades o tiempos de ejecución según las condiciones definidas. Además, se implementó un controlador basado en una máquina de estados finitos (FSM) que permite al robot orientarse y desplazarse hacia cada punto de manera secuencial utilizando control en lazo abierto.

Asimismo, se consideraron aspectos como la calibración de los motores, la identificación de la zona lineal de operación (dead zone y saturación), y la compensación de errores mediante factores de ajuste. Finalmente, se validó el funcionamiento del sistema mediante pruebas en el Puzzlebot, verificando el seguimiento de las trayectorias dadas.

## Objetivos

El objetivo principal es desarrollar un sistema de navegación multipunto para el Puzzlebot mediante el uso de ROS 2, implementando un controlador en lazo abierto basado en una máquina de estados finitos (FSM) que permita al robot seguir trayectorias definidas por el usuario a través de comandos de velocidad.

- Implementar un generador de trayectorias que permita definir múltiples puntos (waypoints) para el desplazamiento del robot.
- Calcular la distancia y el ángulo entre puntos consecutivos para determinar el movimiento requerido del robot.
- Estimar velocidades o tiempos de ejecución según la entrada del usuario, considerando restricciones del sistema.
- Publicar la trayectoria mediante un mensaje personalizado en el tópico `/pose`.
- Desarrollar un controlador basado en máquina de estados finitos (FSM) para orientar y desplazar el robot hacia cada punto de manera secuencial.
- Publicar comandos de velocidad en el tópico `cmd_vel` utilizando mensajes de tipo `Twist`.
- Identificar la zona lineal de operación de los motores (dead zone y saturación) mediante pruebas experimentales.
- Implementar factores de calibración para compensar errores en el desplazamiento lineal y angular.

## Introducción

En este reto se aborda la navegación multipunto de un robot móvil utilizando ROS 2, mediante la implementación de un control en lazo abierto. El sistema permite que el robot siga una trayectoria definida por varios puntos, utilizando mensajes tipo

`Twist` para controlar su movimiento. Para lograrlo, se emplea el cálculo de distancias y ángulos entre puntos, así como una máquina de estados finitos (FSM) que manda las acciones del robot, como girar o avanzar. Este enfoque permite comprender el comportamiento de un robot móvil y las limitaciones del control sin retroalimentación.

**Mensajes**

Los *msg files* son archivos de texto simples para especificar la estructura de datos de un mensaje. Estos archivos se almacenan en el subdirectorio `msg` de un paquete [1]. Un nodo envía o recibe datos entre nodos a través de un mensaje por lo cual estas estructuras de datos específicas permiten a los nodos comunicarse entre sí mediante el modelo publicador/suscriptor [2]. Los mensajes son variables como enteros, puntos flotantes y booleanos. La estructura de mensajes anidados que contiene otros mensajes o una matriz de mensajes se puede utilizar en el mensaje [2].

**Tipos de mensajes**

Los tipos de mensajes siguen las convenciones de nomenclatura estándar de ROS: el nombre del paquete + / + el nombre del archivo .msg. Por ejemplo, std_msgs/msg/String.msg tiene el tipo de mensaje std_msgs/String [1]. Además del nombre del paquete y del archivo, ROS utiliza un identificador único denominado suma MD5 para el control de versiones de los mensajes. Este código actúa como un ID único del archivo .msg, asegurando que la estructura de datos sea idéntica en ambos extremos. Para garantizar la integridad del sistema, dos nodos sólo pueden establecer comunicación si coinciden exactamente tanto el tipo de mensaje como su suma MD5, evitando así errores por incompatibilidad de formatos. [1].

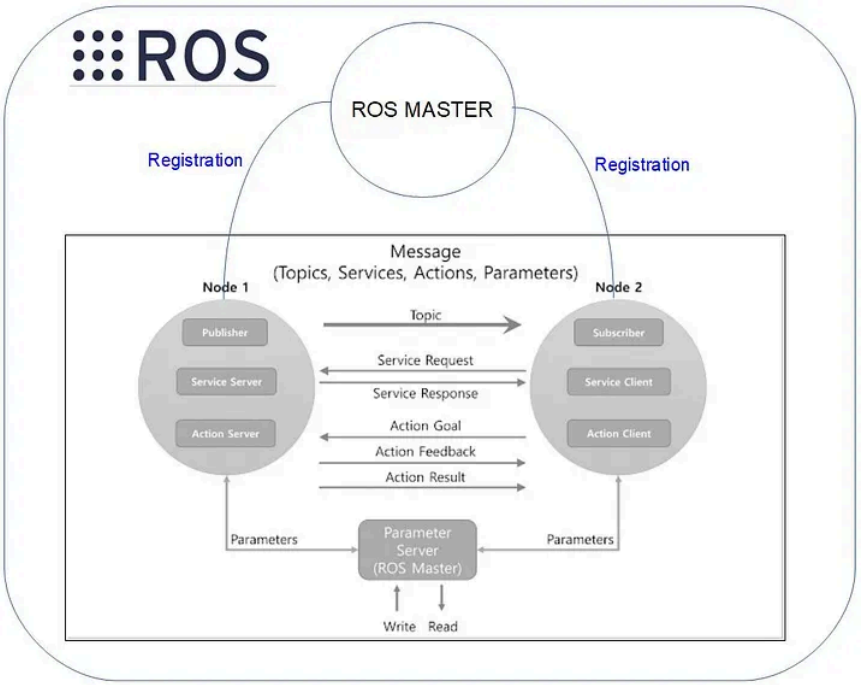



**Figura 1:** Esquematico que muestra el funcionamiento de los mensajes.
https://medium.com/@robtech.impaciente/ros-robot-operating-system-fundamentos-e92478c26e02

**Mensajes Pose**

Los mensajes tipo `/pose` representan la posición y orientación de un robot en el espacio. Generalmente incluyen coordenadas en el plano (x, y) y un ángulo de orientación (θ), permitiendo definir la ubicación de un robot o puntos objetivos dentro de una trayectoria. Estos mensajes son utilizados para describir los puntos que el robot debe alcanzar durante su desplazamiento [3]. Durante este reto se integra la trayectoria, es decir los puntos a recorrer dentro de este mensaje.

**Mensajes Twist**

Los mensajes tipo `/Twist` se utilizan para controlar el movimiento del robot, ya que contienen las velocidades lineales y angulares. En particular, la velocidad lineal (en el eje x) permite el avance o retroceso del robot, mientras que la velocidad angular (alrededor del eje z) controla la rotación. Estos mensajes son publicados en el tópico `/cmd_vel`, el cual es el principal medio de control de robots móviles en ROS 2 [4].

**Mensajes Points**

Los mensajes tipo `/Point` se utilizan para representar coordenadas en el espacio, generalmente en tres dimensiones (x, y, z). En el contexto de este proyecto, se emplean principalmente para definir los puntos (waypoints) que conforman la trayectoria que el robot debe seguir. Cada punto indica una posición objetivo en el plano, donde el robot debe desplazarse de manera secuencial [5].

Estos mensajes son especialmente útiles para construir trayectorias, ya que permiten almacenar una lista ordenada de posiciones que el robot recorrerá. En este caso, los mensajes Point se integran dentro de un mensaje personalizado de tipo trayectoria, facilitando la comunicación entre el generador de trayectorias y el controlador. De esta forma, el robot puede interpretar cada punto como una meta intermedia y calcular el movimiento necesario para alcanzarla.

**Lazo abierto de control en un robot móvil**

En un sistema de lazo abierto, la salida no tiene efecto sobre la acción de control; es decir, la señal de salida no se mide ni se retroalimenta para compararla con la entrada. Por ello, la precisión del sistema depende enteramente de la calibración, funcionando bajo condiciones de operación fijas (como los sistemas basados en temporizadores). Estos sistemas, también llamados de alimentación directa (feedforward), solo son efectivos si no existen perturbaciones externas y si la relación entre la entrada y la salida se conoce con exactitud [6].

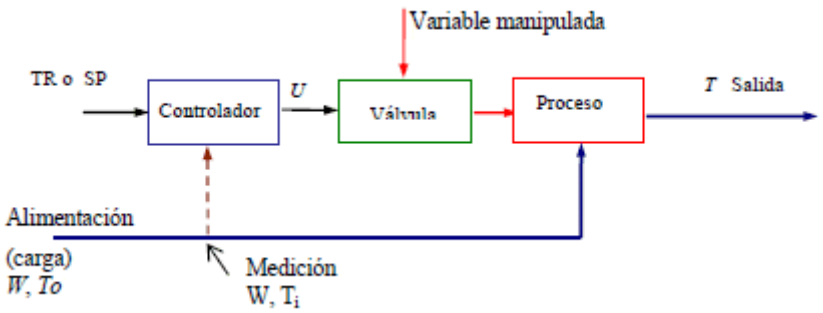



**Figura 2:** Diagrama de bloques del sistema de control de lazo abierto.
https://medium.com/@robtech.impaciente/ros-robot-operating-system-fundamentos-e92478c26e02

En este caso, el robot calcula previamente el tiempo o las velocidades necesarias para moverse entre puntos, sin corregir errores durante la ejecución. Aunque es sencillo de implementar, este tipo de control puede verse afectado por perturbaciones, errores en los actuadores o imprecisiones del modelo como se mencionó previamente.

**Cálculo de la distancia y ángulo recorrido por un robot móvil.**

El cálculo de distancia y ángulo entre dos puntos es esencial para la navegación. La distancia se obtiene mediante la fórmula euclidiana, que mide la distancia entre dos puntos:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

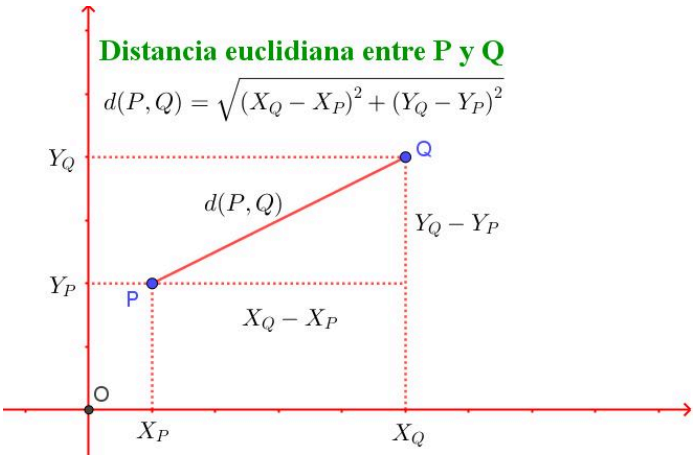
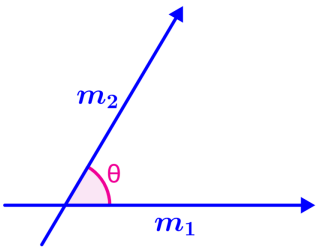



**Figura 3:** Imagen mostrando el como se obtiene la distancia entre dos puntos.
https://medium.com/@robtech.impaciente/ros-robot-operating-system-fundamentos-e92478c26e02

Mientras que el ángulo de orientación se calcula con la función siguiente:

$$\theta = arctan2(y_2 - y_1, x_2 - x_1)$$




$$\tan(\theta) = \frac{m_2 - m_1}{1 + m_2 m_1}$$

**Figura 4:** Imagen mostrando como se obtiene el angulo entre dos puntos.
https://www.neurochispas.com/wiki/angulo-entre-dos-rectas-formulas-y-ejercicios/

Estos cálculos permiten determinar cuánto debe avanzar y girar el robot para alcanzar cada punto de la trayectoria.

**Solución del problema**

1. **Identificación de la región lineal de los motores**

Para un correcto funcionamiento del Puzzlebot, se realizó el proceso para conocer la variable mínima de funcionamiento y la máxima (dead zone y saturación respectivamente), esto para que los motores del Puzzlebot operaran en su región lineal; y, además, porque el conocimiento de la velocidad máxima era fundamental para, más adelante, poder determinar si una trayectoria era posible o no en un tiempo determinado.

Para esto, se utilizó la herramienta `teleop_twist_keyboard`, la cual proporciona una interfaz interactiva que permite controlar el movimiento del robot mediante el teclado de la computadora, facilitando el ajuste de la velocidad en tiempo real. Mediante el uso de esta herramienta, se inició con los motores completamente detenidos y se incrementó gradualmente la velocidad lineal. A partir de este procedimiento, se identificó el punto en el cual los motores comienzan a moverse, lo que permitió determinar la zona muerta del sistema. Como resultado, se observó que, para superar la fricción inicial y generar movimiento en los motores, es necesario aplicar una velocidad lineal en el eje x de al menos **0.000808**.

Por el otro lado, para encontrar la velocidad máxima a la que los motores pueden moverse sin que haya un exceso de corriente provocada por el movimiento, se fue aumentando la velocidad poco a poco hasta que los motores no incrementaron más su velocidad. Se obtuvo que la velocidad máxima a la que pueden llegar es **0.742**.

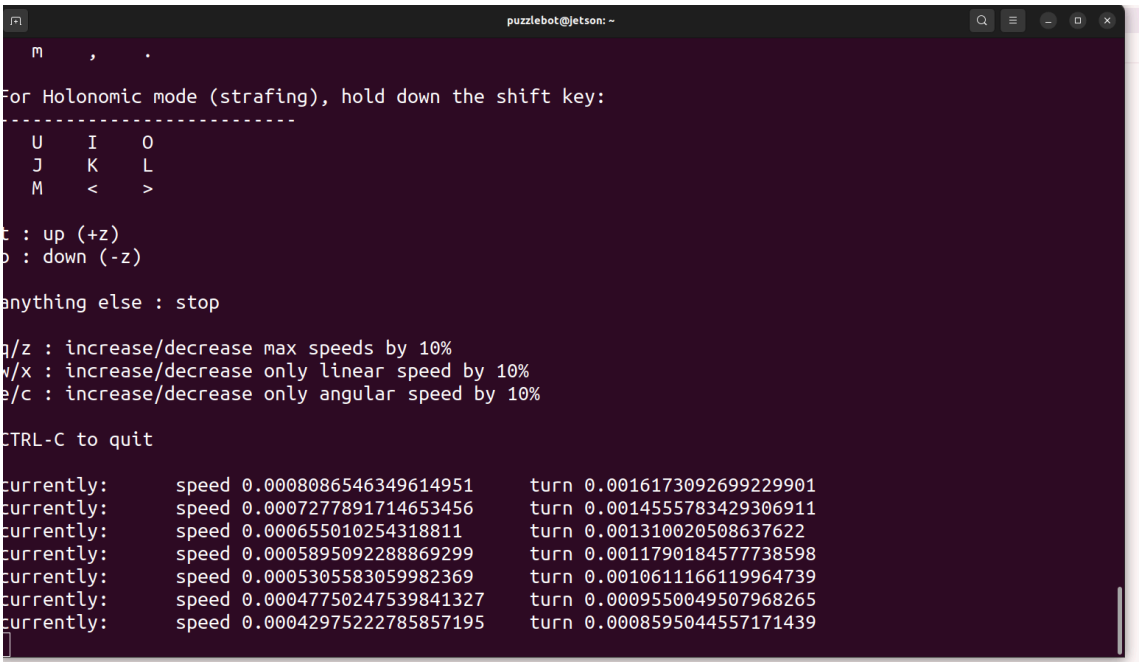



**Figura 5:** Imagen de la terminal mostrando la zona muerta de los motores (velocidad mínima).

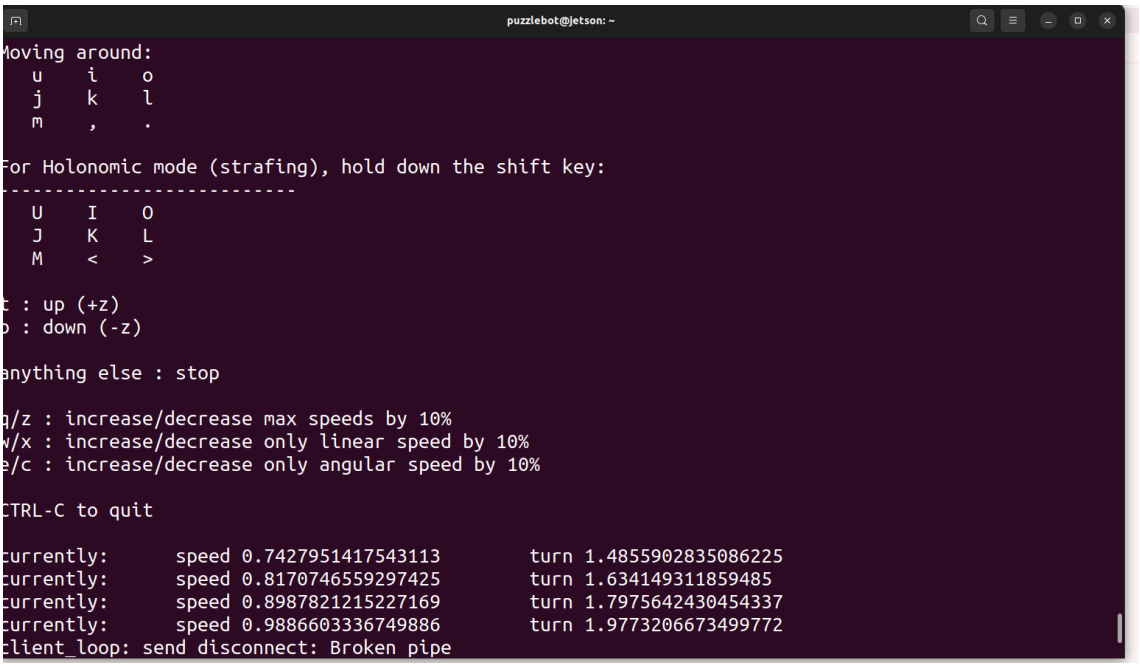



**Figura 6:** Imagen de la terminal mostrando la zona de saturación de los motores (velocidad máxima).

## 2. Calibración en la superficie de pruebas

Para mejorar la precisión del sistema en lazo abierto, se realizó una calibración experimental de los movimientos lineales y angulares del robot sobre la superficie de pruebas. Inicialmente, se utilizaron los valores teóricos basados en las velocidades definidas:

```
self.linear_speed = 0.2  # m/s
self.angular_speed = 0.5  # rad/s
```

Sin embargo, se observó que el robot presentaba errores en su desplazamiento:

- En movimiento lineal, el robot recorría aproximadamente 2.02 m en lugar de 2.00 m.
- En rotación, el robot presentaba un error aproximado de +10° al intentar girar 90°.

Para corregir estos errores, se utilizó una regla de tres, ajustando los tiempos de ejecución hasta obtener un comportamiento más preciso. Como resultado, se introdujeron factores de calibración en el cálculo del tiempo:

```
self.forward_time = (2.0 / self.linear_speed) * 0.934579
self.rotate_time = ((3.1416/2) / self.angular_speed) * 0.89
```


```
self.forward_time = (2.0 / self.linear_speed) * 0.934579
self.rotate_time = ((3.1416/2) / self.angular_speed) * 0.89
```


Los resultados de calibración se muestran a continuación:

| Prueba | Distancia esperada (m) | Distancia medida |
|---|---|---|
| 1 | 2 m | 2.02m |
| 2 | | 2.02m |
| 3 | | 2.01m |
| 4 | | 2.02m |
| 5 | | 2.02m |

**Tabla 1:** Muestras de la distancia de calibración.

| Prueba | Ángulo esperado (°) | Ángulo medido (°) |
|---|---|---|
| 1 | 90 ° | 100° |
| 2 | | 99° |
| 3 | | 99° |
| 4 | | 100° |
| 5 | | 101° |

**Tabla 2:** Muestras de los ángulos de calibración.

3. **Modificación de máquina de estados para prueba 1 (cuadrado)**

Utilizando como base el controlador en lazo abierto proporcionado por Manchester Robotics, se realizaron las modificaciones necesarias para lograr que el Puzzlebot pudiera seguir la trayectoria de un cuadrado de 2 x 2 m.

El código define un nodo de ROS2 cuyo objetivo es seguir la trayectoria de un cuadrado, fijando la velocidad y realizando los cálculos con respecto al tiempo. El controlador se basa en una máquina de estados (FSM) y está diseñado para que el robot avance 4 tramos rectos de 2 m y realice 4 giros de 90° para poder realizar la trayectoria.

En el constructor del nodo se establece como estado base el 0 que equivale a "forward", y se fijan las velocidades lineales y angulares, de 0.2 m/s y 0.5 rad/s respectivamente. Con base en esto, se realizan los cálculos del tiempo que necesita el Puzzlebot para avanzar 2 m y para rotar 90°:

```
self.forward_time = (2.0 / self.linear_speed)*0.934579
self.rotate_time = ((3.1416/2) / self.angular_speed)*0.89
self.backward_time = self.forward_time
```

Adicionalmente, a los tiempos se les agregó el factor de calibración calculado previamente, lo que permitió añadir robustez para una superficie determinada. Además, el estado se actualiza con una frecuencia de 10 Hz.

La máquina de estados funciona de la siguiente manera:

- Se actualiza el tiempo actual y el tiempo transcurrido.
- Se crea una variable de tipo `Twist`.
- Se evalúa el estado actual.
    - Si el estado actual es 0 (Avanzar), se asigna la velocidad lineal al componente lineal x del mensaje `Twist`. Se evalúa si el tiempo transcurrido, es mayor al tiempo de avance; si es el caso, se pasa al estado 1 (girar).
    - Si el estado actual es 1 (Rotar) se asigna la velocidad angular al componente angular z del mensaje `Twist`. Se evalúa si el tiempo transcurrido, es mayor al tiempo de avance; si es el caso, se pasa al estado 0 (girar).
    - Los estados 2, 4 y 6 funcionan funcionan igual que el estado 0, permitiendo que el Puzzlebot recorra aproximadamente 2 metros.
    - Los estados 3, 5 y 7 funcionan igual que el estado 1, permitiendo que el Puzzlebot rote aproximadamente 90°.
    - El estado 7 permite cambiar al estado 8 (Detenerse) que se ejecuta mediante un `else` que asigne velocidad lineal y angular 0.
    - Para cualquier cambio de estado, se actualiza el tiempo en el que inicia el estado actual para poder calcular el tiempo transcurrido.
- Se publica el mensaje tipo `Twist` al tópico `/cmd_vel`.

## 4. Interfaz para el envío de mensajes personalizados

Se desarrolló una interfaz dentro del nodo generador de trayectorias que permite al usuario definir una ruta personalizada mediante dos modalidades: la entrada manual por consola o mediante archivos de parámetros en formato YAML Esta interfaz

construye un mensaje personalizado que es enviado al controlador a través del tópico `/pose`.

```
n = int(input("¿Cuántos puntos tiene la trayectoria?: "))
points_list = []
for i in range(n):
    p = Point()
    p.x = float(input("X: "))
    p.y = float(input("Y: "))
    p.z = 0.0
    points_list.append(p)
    msg.points.append(p)
```

En la implementación final se prioriza la entrada manual por consola, permitiendo mayor modularidad

Con base en esta información, el sistema construye y publica un mensaje personalizado que contiene la trayectoria completa y los parámetros de movimiento. Posteriormente, se construye y publica un mensaje personalizado en el tópico `/pose`, permitiendo la comunicación con el controlador:

```
self.publisher_.publish(msg)
```

Para los mensajes tipo `/Pose` personalizados se utilizó un mensaje personalizado `Trajectories`, el cual contiene la información necesaria para la navegación del robot, incluyendo los puntos de la trayectoria y las velocidades.

```
msg = Trajectories()
msg.points.append(p)
...
msg.linear_velocity = v_lin
      msg.angular_velocity = v_ang
```

En el controlador, este mensaje es recibido y procesado para ejecutar la trayectoria

```
self.subscription = self.create_subscription(
    Trajectories, '/pose', self.path_callback, 10
)
```

El sistema permite definir parámetros clave como los puntos de la trayectoria, velocidades lineales y angulares, así como el tiempo total de ejecución. En el generador de trayectorias, el usuario puede elegir entre definir tiempo o velocidades:

```
opcion = input("¿Enviar por Tiempo (t) o Velocidad (v)?:
").lower()
```

En caso de definir tiempo, se calculan automáticamente las velocidades:

```
t_lineal_restante = t_user - t_rotacion
v_lineal_calc = total_dist / t_lineal_restante
...
msg.linear_velocity = v_lineal_calc
msg.angular_velocity = MAX_ANGULAR_V
```

En caso de definir velocidades, se estima el tiempo total:

```
t_estimado = (total_angle / v_ang) + (total_dist / v_lin)
```

Estos parámetros son enviados al controlador mediante el mensaje personalizado, permitiendo adaptar el comportamiento del robot según las condiciones deseadas.

## 5. FSM para trayectoria personalizada

Para poder realizar la trayectoria personalizada por el usuario, se realizó un nodo cuyo funcionamiento se basa en el diagrama proporcionado por Manchester Robotics de una FSM, el cual se observa en la figura 7.

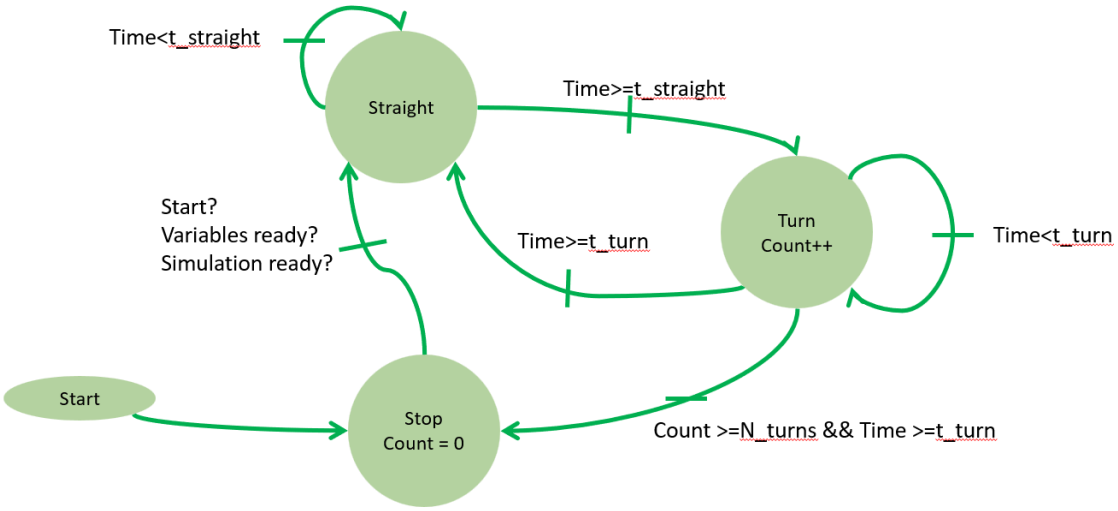



**Figura 7:** Diagrama de una FSM como controlador en lazo abierto para una trayectoria.

La implementación de la FSM a nivel código cuenta con tres estados:

- Idle (0), que espera la trayectoria.
- Rotate (1), que orienta al robot hacia el siguiente punto.
- Move (2), que desplaza el robot hacia el punto objetivo.

Estos estados se declaran en el constructor del nodo, en donde además se establece como estado inicial el 0 (idle). En el constructor también se crea el suscriptor al tópico `/pose` y el publicador al tópico `/cmd_vel`. Asimismo, se declaran los factores de calibración obtenidos para una determinada superficie, y se declaran las variables de velocidad lineal y angular, así como los puntos, y la pose (asumiendo que el inicio es en 0, 0, 0; es decir, en el origen con orientación hacia 0°).

Para el funcionamiento también se declara la función `path_callback`, que recibe el mensaje personalizado tipo `Trajectories` y "descomprime" este mensaje, permitiendo obtener los puntos de la trayectoria, así como la velocidad lineal y angular asignadas. Además, cuando se recibe la trayectoria, también se cambia el estado a "rotate" y se actualiza el tiempo de inicio del estado, además de asignar el índice actual a 0.

En el loop del controlador primero se evalúa si el estado actual es IDLE, en cuyo caso ocurre un `return` para continuar esperando el mensaje tipo `Trajectories`

mediante el suscriptor a `/pose`. En caso contrario, se obtiene el tiempo actual y el tiempo transcurrido desde el último cambio de estado; además se crea un mensaje tipo `Twist()` para el publicador al tópico `/cmd_vel`. Asimismo, se obtiene el punto objetivo mediante el índice actual, y se extraen las coordenadas (x,y) para calcular la diferencia y con esto la distancia entre el punto actual y el punto objetivo. Asimismo se calcula el ángulo hacia el objetivo y el ángulo que debe de rotar.

```
now = self.get_clock().now()
elapsed = (now - self.state_start_time).nanoseconds * 1e-9
cmd = Twist()

target = self.points[self.current_idx]
dx = target.x - self.cur_x
dy = target.y - self.cur_y
dist = np.sqrt(dx**2 + dy**2)
target_angle = np.arctan2(dy, dx)
angle_to_turn = (target_angle - self.cur_theta + np.pi) % (2 * np.pi) - np.pi
```

Posteriormente se evalúa el estado actual:

- En el caso de `ROTATE`, se calcula la duración de la rotación con base en la velocidad angular, y se multiplica por el factor de calibración para la velocidad angular.
    - Si el tiempo transcurrido durante el estado es menor a la duración del estado, se asignará dicha velocidad angular en z (positiva o negativa dependiendo del ángulo a rotar) para el tópico `/cmd_vel`.
    - Una vez el tiempo transcurrido sea mayor a la duración calculada, se actualiza el ángulo actual como el ángulo objetivo respecto al sistema de referencia, y se cambia al estado `MOVE`.
- En el caso de `MOVE`, se calcula la duración de la rotación con base en la velocidad angular, y se multiplica por el factor de calibración para la velocidad lineal.
    - Si el tiempo transcurrido durante el estado es menor a la duración del estado, se asignará dicha velocidad lineal en x para el tópico `/cmd_vel`.
    - Una vez el tiempo transcurrido sea mayor a la duración calculada, se actualizan las coordenadas (x, y). Además se actualiza el índice del punto alcanzado. Si el índice es menor a la cantidad de puntos, se cambia al estado `ROTATE`; en caso contrario, significa que se ha alcanzado el destino, por lo que se cambia al estado `IDLE`.

Además, se utilizan diferentes loggers como referencia para el usuario, ya que los mensajes en la terminal permiten comprender con claridad qué parte del proceso se está ejecutando. Finalmente, se publica el mensaje `Twist` al tópico `/cmd_vel` y se repite el proceso del loop.

**Resultados**

Se realizaron distintas pruebas del sistema utilizando el Puzzlebot con el objetivo de validar el correcto funcionamiento de este. Inicialmente se verificó usando

el archivo YAML, donde se establece los puntos previamente, el sistema fue capaz de interpretar correctamente estos valores y construir la trayectoria deseada, sin embargo, nos dimos cuenta que cambiar el archivo de parámetros YAML era más complicado para la interacción del usuario; por lo tanto los ejemplos que se muestran a continuación se usa principalmente inputs por consola, como se muestra en la figura 8.

```
puzzlebot@jetson:~$ ros2 run challenge2 path_generator
[INFO] [1705499319.801204511] [path_generator]: Nodo Generador (Cerebro) iniciado.

¿Cuántos puntos tiene la trayectoria?: 3
--- Punto 1 ---
X: -1
Y: 2
--- Punto 2 ---
X: -2
Y: 0
--- Punto 3 ---
X: 0
Y: 0

Distancia total: 6.47m | Giro total: 6.28 rad
¿Enviar por Tiempo (t) o Velocidad (v)?: v
Introduce velocidad lineal (máx 0.7): 0.5
Introduce velocidad angular (máx 0.5): 0.3
El robot tardará aproximadamente 33.89s en total.
[INFO] [1705499342.171564711] [path_generator]: Puntos seleccionados: [(-1.0, 2.0), (-2.0, 0.0), (0.0, 0.0)]

Cierra la ventana de la gráfica para enviar la trayectoria al robot...
[INFO] [1705499348.038437157] [path_generator]: Trayectoria validada y enviada al controlador.
puzzlebot@jetson:~$ ros2 run challenge2 path_generator
```

**Figura 8:** Loggers utilizados para interacción del usuario mediante consola.

Además, se implementó una visualización mediante gráficas, lo que permitió validar de manera previa la forma de la trayectoria antes de ser ejecutada por el robot como se muestra a continuación.

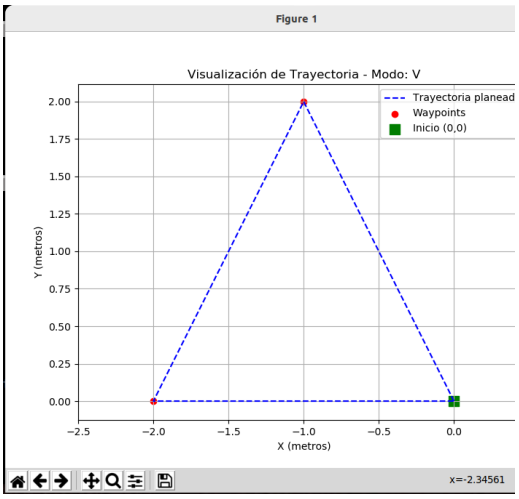



**Figura 9:** Visualización de la trayectoria triangular calculada.

### Trayectoria 1: Triángulo

En la figura 9 se presenta el seguimiento de una trayectoria triangular definida por los puntos de paso (waypoints) (0,0), (-2,0) y (-1,2). Para su ejecución, se utilizó el modo de control por velocidad, estableciendo una velocidad lineal constante de 0.5 m/s y una velocidad angular de 0.3 rad/s. Los resultados obtenidos se detallan a continuación:

```
puzzlebot@jetson:~$ ros2 run challenge2 path_generator
[INFO] [1705499319.801204511] [path_generator]: Nodo Generador (Cerebro) iniciado.

¿Cuántos puntos tiene la trayectoria?: 3
--- Punto 1 ---
X: -1
Y: 2
--- Punto 2 ---
X: -2
Y: 0
--- Punto 3 ---
X: 0
Y: 0

Distancia total: 6.47m | Giro total: 6.28 rad
¿Enviar por Tiempo (t) o Velocidad (v)?: v
Introduce velocidad lineal (máx 0.7): 0.5
Introduce velocidad angular (máx 0.5): 0.3
El robot tardará aproximadamente 33.89s en total.
[INFO] [1705499342.171564711] [path_generator]: Puntos seleccionados: [(-1.0, 2.0), (-2.0, 0.0), (0.0, 0.0)]

Cierra la ventana de la gráfica para enviar la trayectoria al robot...
[INFO] [1705499348.038437157] [path_generator]: Trayectoria validada y enviada al controlador.
puzzlebot@jetson:~$ ros2 run challenge2 path_generator
```

**Figura 10:** Output en terminal mostrando la configuración de la trayectoria 1.

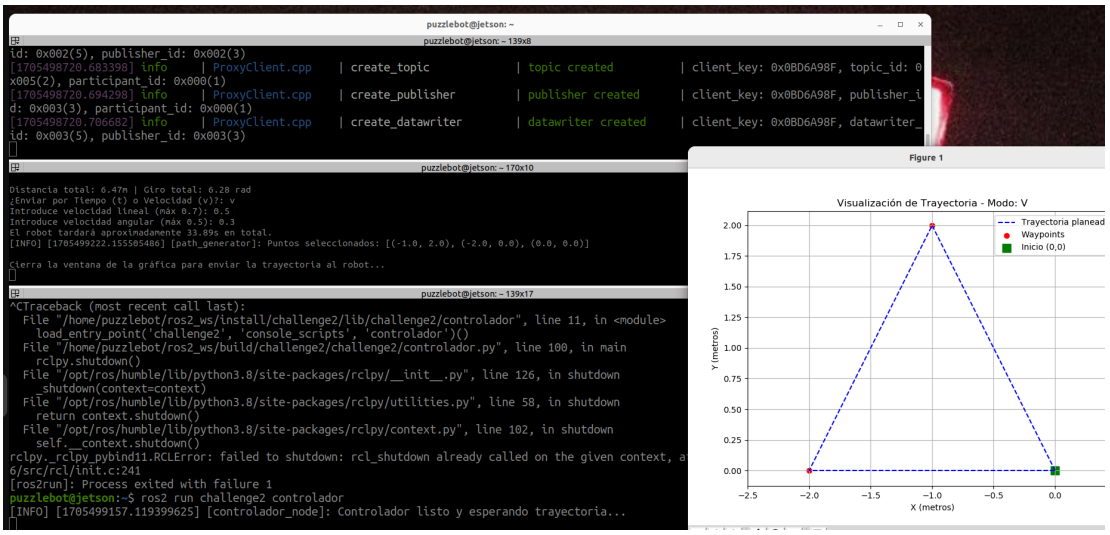



**Figura 11:** Output mostrando la trayectoria 1 y su ejecución completa.

Link del video demostrativo:
https://drive.google.com/file/d/1daBvWaQYnzKYeYMZikg95VtsxpsVtcNd/view?usp=drive_link

Como se observa en las figuras 10 y 11, así como en el video adjunto, la trayectoria se completa de manera satisfactoria y con precisión. Este desempeño es resultado de las calibraciones realizadas previamente; no obstante, se presenta un ligero desfase en el recorrido final, el cual es atribuible al error acumulado característico de los sistemas de lazo abierto.

**Trayectoria 2: Ejemplo de Manchester Robotics**

En la figura 12 se presenta el seguimiento de una trayectoria definida por Manchester Robotics:

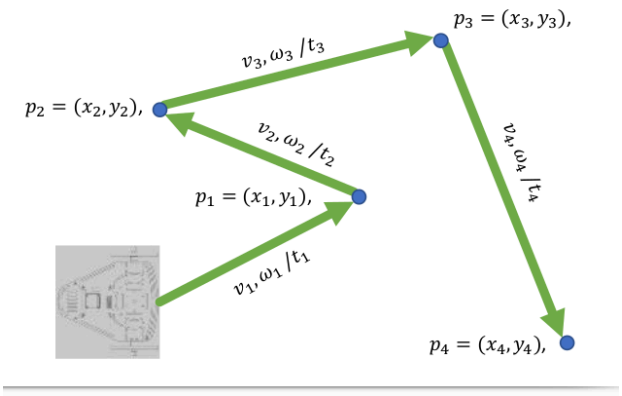



**Figura 12:** Trayectoria de ejemplo (Manchester Robotics, s.f.).

Esta trayectoria se define por los puntos de paso (waypoints) (1,1), (0,2), (1.5,2.5), y (2.0,-0.5). Para su ejecución, se utilizó el modo de control por velocidad, estableciendo una velocidad lineal constante de 0.4 m/s y una velocidad angular de 0.3 rad/s. Los resultados obtenidos se detallan a continuación:

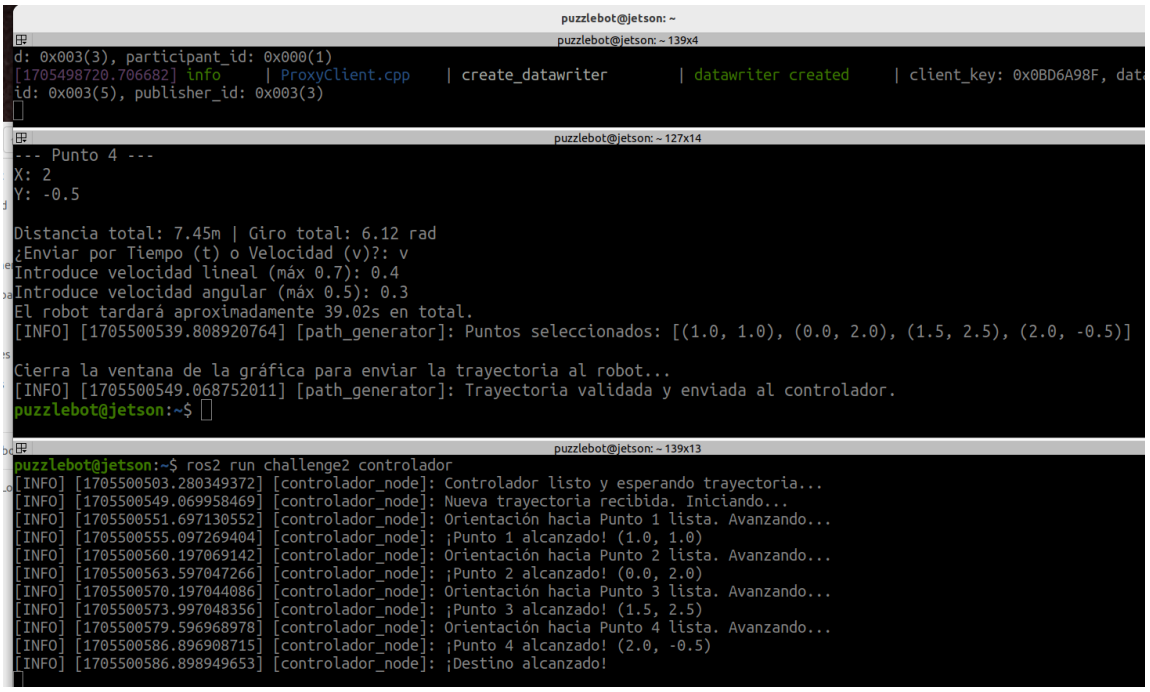



**Figura 13:** Output en terminal mostrando la configuración de la trayectoria 2.

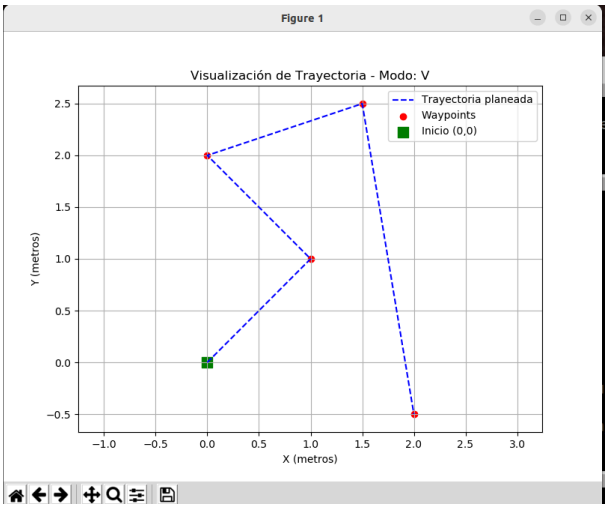



**Figura 14:** Visualización de la trayectoria 2 calculada.

Link del video demostrativo:
https://drive.google.com/file/d/1CF4wtcegz6chl-tjc6jbfWvTuBcvboV-/view?usp=drive_link

Como se observa en las figuras 13 y 14, así como en el video adjunto, la trayectoria se completa de manera satisfactoria y con precisión.

**Trayectoria 3: Rombo**

En la figura 15 se presenta el seguimiento de un rombo definido por los puntos de paso (waypoints) (1,1), (2,0), (1,-1) y (0,0). Para su ejecución, se utilizó el modo de control por tiempo, estableciendo un tiempo de 25s. Los resultados obtenidos se detallan a continuación:

```
puzzlebot@jetson: ~ 127x14
X: 1
Y: -1
--- Punto 4 ---
X: 0
Y: 0

Distancia total: 5.66m | Giro total: 5.50 rad
¿Enviar por Tiempo (t) o Velocidad (v)?: t
Introduce el tiempo TOTAL deseado (segundos): 25
[INFO] [1705500158.325994035] [path_generator]: Puntos seleccionados: [(1.0, 1.0), (2.0, 0.0), (1.0, -1.0), (0.0, 0.0)]

Cierra la ventana de la gráfica para enviar la trayectoria al robot...
[INFO] [1705500176.675310747] [path_generator]: Trayectoria validada y enviada al controlador.
puzzlebot@jetson:~$
```

```
puzzlebot@jetson: ~ 139x13
puzzlebot@jetson:~$ ros2 run challenge2 controlador
[INFO] [1705500111.086930980] [controlador_node]: Controlador listo y esperando trayectoria...
[INFO] [1705500176.676142361] [controlador_node]: Nueva trayectoria recibida. Iniciando...
[INFO] [1705500178.202562361] [controlador_node]: Orientación hacia Punto 1 lista. Avanzando...
[INFO] [1705500181.602584234] [controlador_node]: ¡Punto 1 alcanzado! (1.0, 1.0)
[INFO] [1705500184.702547410] [controlador_node]: Orientación hacia Punto 2 lista. Avanzando...
[INFO] [1705500188.102663294] [controlador_node]: ¡Punto 2 alcanzado! (2.0, 0.0)
[INFO] [1705500191.202481522] [controlador_node]: Orientación hacia Punto 3 lista. Avanzando...
[INFO] [1705500194.602618604] [controlador_node]: ¡Punto 3 alcanzado! (1.0, -1.0)
[INFO] [1705500197.702817562] [controlador_node]: Orientación hacia Punto 4 lista. Avanzando...
[INFO] [1705500201.102585789] [controlador_node]: ¡Punto 4 alcanzado! (0.0, 0.0)
[INFO] [1705500201.104633133] [controlador_node]: ¡Destino alcanzado!
```

**Figura 15:** Output en terminal mostrando la configuración de la trayectoria 3.

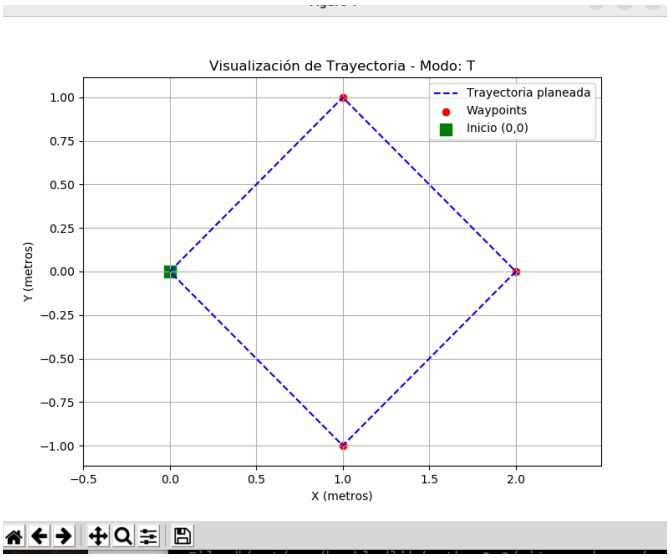



**Figura 16:** Visualización de la trayectoria 3 calculada.

Link del video demostrativo:
https://drive.google.com/file/d/178QzPmNxgJZOhfDHfAArG4nF8Ikvua7a/view?usp=drive_link

Como se observa en las figuras 15 y 16, así como en el video adjunto, la trayectoria se completa de manera satisfactoria y con precisión.

**Trayectoria 4: Semi cuadrado**

En la figura 17 se presenta el seguimiento de un semi cuadrado definido por los puntos de paso (waypoints) (1,1), (1,0), (0,0) y (0,1). Para su ejecución, se utilizó el modo de control por tiempo, estableciendo un tiempo de 25s. Los resultados obtenidos se detallan a continuación:

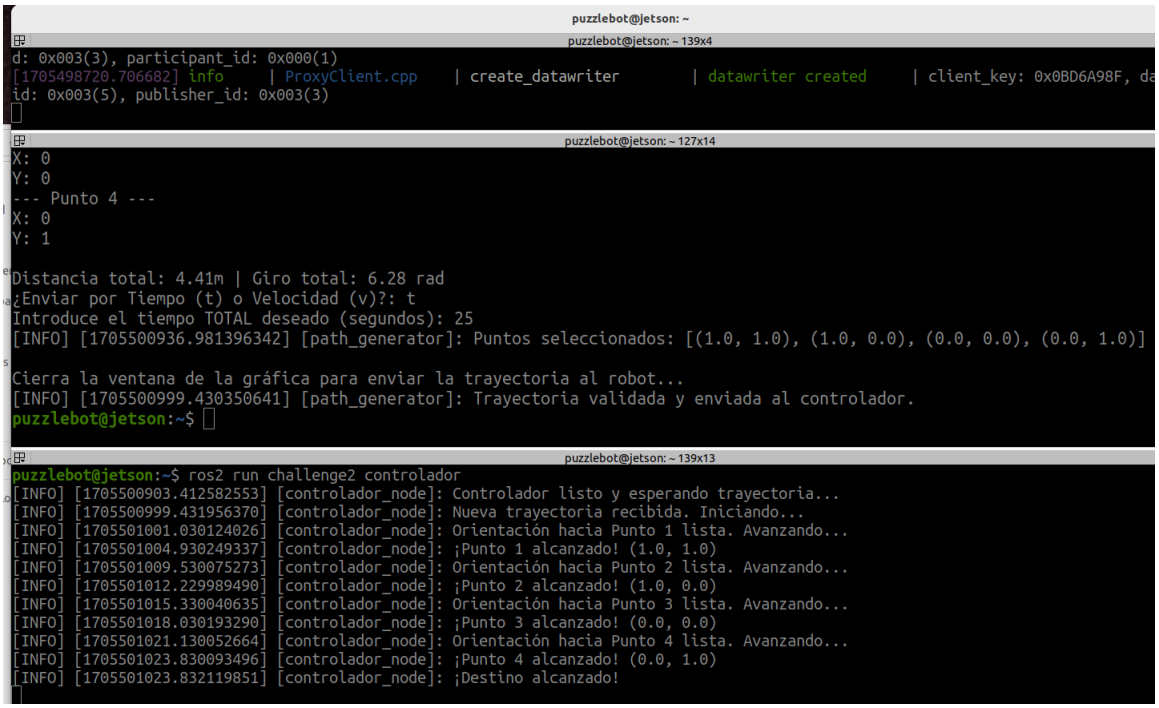



**Figura 17:** Output en terminal mostrando la configuración de la trayectoria 4.

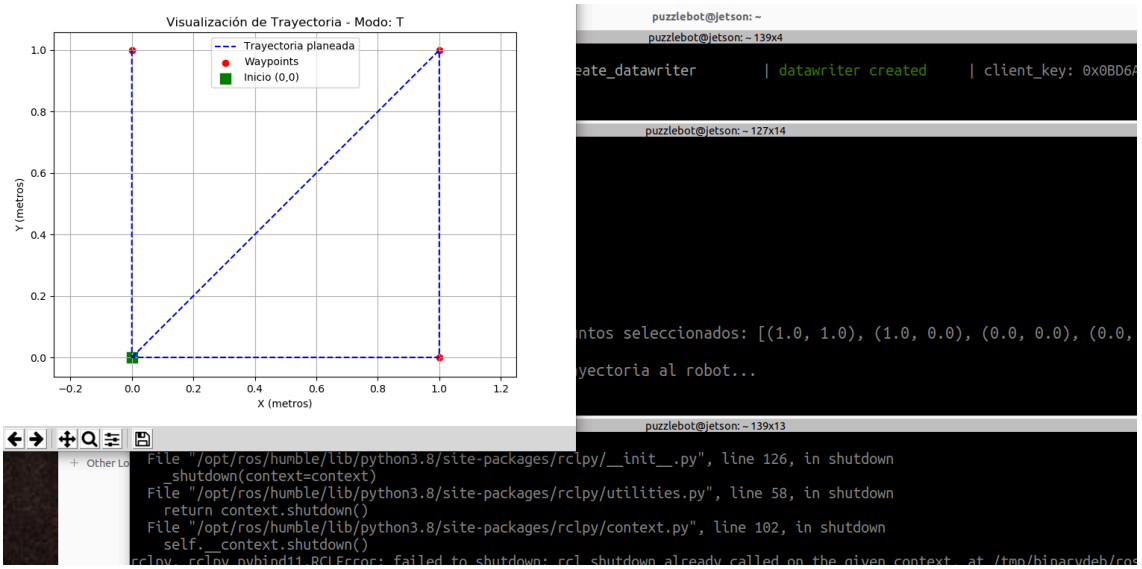



**Figura 18:** Visualización de la trayectoria 4 calculada.

Link del video demostrativo:
https://drive.google.com/file/d/1itnncyfboOHdLBTmkpepSPqKOvHnSX4v/view?usp=drive_link

Como se observa en las figuras 17 y 18, así como en el video adjunto, la trayectoria se completa de manera satisfactoria y con precisión.

Carpeta con los videos demostrativos :
https://drive.google.com/drive/folders/1NHHBFXzu2D8efQtvCI4d6P-RdNr-duso?usp=sharing

Por otro lado, se implementó la primera trayectoria propuesta por Manchester mediante un controlador de lazo abierto. Dicho control se basa en una Máquina de Estados Finitos (FSM), la cual ejecuta una rutina de ocho estados para completar el recorrido. La figura se muestra a continuación:

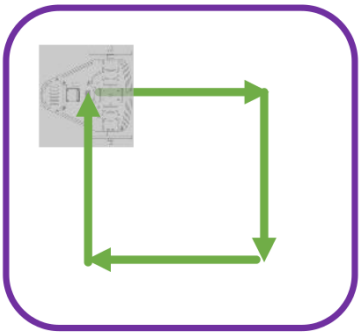


**Figura 19:** Visualización de la trayectoria cuadrada.

El video haciendo la rutina obtenida se muestra a continuación: https://drive.google.com/file/d/1ZQbZv3p4pNwvRudJRXv0ioSy9mg12dX9/view?usp=drive_link

**Conclusiones**

Se lograron los objetivos establecidos en este reto debido a que fue posible establecer una trayectoria al Puzzlebot de al menos 4 puntos, verificando si la velocidad o el tiempo eran posibles. Además, el algoritmo a pesar de que fue a lazo abierto, tenía cierta robustez debido a que previamente se calibraron los rangos de velocidad lineal y angular mediante un método heurístico, de esta manera se encontraron factores de calibración para los datos que se mandaban al tópico de `/cmd_vel`, de esta manera, a pesar de que no hubo retroalimentación de ningún sensor, las trayectorias establecidas por el usuario fueron lo más exactos posibles, a su vez, se utilizó la información que se recabó del reto pasado de la zona muerta y la velocidad máxima del robot para verificar si la velocidad o el tiempo del usuario eran físicamente posibles de realizar. Este reto funcionó como base para más adelante realizar un control a lazo cerrado el cual permitirá tener retroalimentación de los encoders del robot para que realice trayectorias más precisas sin tener que depender de una calibración hecha en condiciones específicas.

**Referencias**

[1] *ROS.org. (s. f.). Messages. ROS Wiki. Recuperado el 11 de abril de 2026, de https://wiki.ros.org/Messages*

[2] *RobTech.* (20 de mayo de 2020). ROS (Robot Operating System). Fundamentos. Medium. https://medium.com/@robtech.impaciente/ros-robot-operating-system-fundamentos-e92478c26e02

[3] *ROS.org.* (s. f.). Pose Message (geometry_msgs/Pose). ROS Documentation (Noetic). Recuperado el 12 de abril de 2026, de https://docs.ros.org/en/noetic/api/geometry_msgs/html/msg/Pose.html

[4] *ROS.org.* (s. f.). Twist Message (geometry_msgs/Twist). ROS Documentation (Noetic). Recuperado el 12 de abril de 2026, de https://docs.ros.org/en/noetic/api/geometry_msgs/html/msg/Twist.html

[5] *ROS.org.* (s. f.). Point Message (geometry_msgs/Point). ROS Documentation (Noetic). Recuperado el 12 de abril de 2026, de https://docs.ros.org/en/noetic/api/geometry_msgs/html/msg/Point.html

[6] Instrumentación y Control. (12 de septiembre de 2017). Sistema de control de lazo abierto (Open-loop). https://instrumentacionycontrol.net/sistema-de-control-de-lazo-abierto-open-loop/